

Gradient naturel et K-FAC

D'un modèle gaussien à K-FAC sur un transformeur

Shayck Cassim

Août-Septembre 2026

Code et notebooks : <https://github.com/cassimshayck/mini-gpt>

1 Motivation

La descente de gradient met à jour les paramètres par $\theta \leftarrow \theta - \eta \nabla L$. Le problème est que cela implique un déplacement de taille η partout, quel que soit le paramètre et quel que soit l'endroit où l'on se trouve. Autrement dit, elle mesure les distances avec la métrique euclidienne sur l'espace des paramètres. Donc un déplacement d'un écart-type de 100 à 100,1 représente le même déplacement que 0,1 à 0,2 ici dans le cas gaussien par exemple.

Or les paramètres ne sont qu'un étiquetage du modèle. Ce qui compte n'est pas de combien les nombres ont bougé, mais de combien la distribution produite par le modèle a changé. Le gradient naturel [2] remplace donc le pas euclidien par

$$\theta \leftarrow \theta - \eta F(\theta)^{-1} \nabla L,$$

où F est la matrice d'information de Fisher, covariance du score $s = \nabla_{\theta} \log p_{\theta}(y | x)$:

$$F(\theta) = \mathbb{E}_{x \sim p_{\text{data}}, y \sim p_{\theta}(\cdot | x)} \left[s s^{\top} \right].$$

F joue le rôle de « pont » entre déplacement de paramètres et changement de distribution : F coïncide avec la Hessienne de la divergence de Kullback-Leibler par rapport à son second argument, évaluée en $d = 0$. C'est ce qui en fait la métrique riemannienne de Fisher-Rao :

$$D_{\text{KL}}(p_{\theta} \parallel p_{\theta+d}) = \frac{1}{2} d^{\top} F(\theta) d + \mathcal{O}(\|d\|^3).$$

L'idée est cependant difficile à mettre en œuvre en pratique : F est de taille $d \times d$ où d est le nombre de paramètres. Pour un transformeur de 10,8 millions de paramètres, elle occuperait environ 460 To en mémoire vive.

L'idée de ce court rapport est d'étudier des modèles assez petits pour que F soit calculable exactement, et de remonter jusqu'à un transformeur où elle ne l'est plus, en s'adaptant à chaque étape.

2 Un modèle gaussien à deux paramètres

L'objectif : Ici, on veut voir si sur un modèle simple on ressent concrètement l'intérêt de la métrique.

Le modèle est $\mathcal{N}(\mu, \sigma^2)$ ajusté par maximum de vraisemblance sur 200 points. La matrice de Fisher est connue en forme close :

$$F = \text{diag} \left(\frac{1}{\sigma^2}, \frac{2}{\sigma^2} \right).$$

On ne détaille pas les calculs, mais il suffit d'intégrer les dérivées partielles secondes de la log-vraisemblance (μ et σ sont orthogonaux pour la métrique de Fisher-Rao).

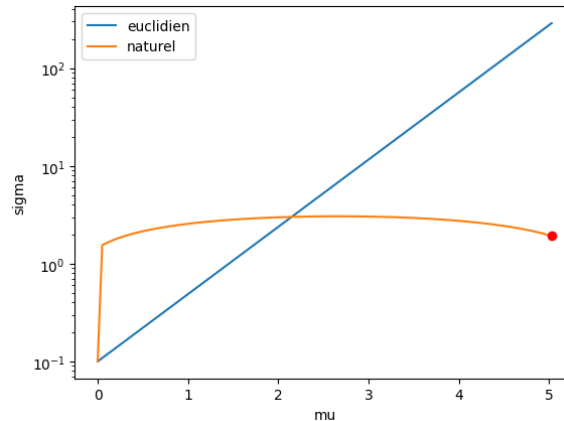


Figure 1: Trajectoires dans le plan (μ, σ) , départ en $(0, 0.1)$, même pas $\eta = 0.01$. Le point rouge est l'optimum.

On remarque ci-dessus que partant d'un écart-type petit, la descente ordinaire projette σ jusqu'à 290 et n'en est pas revenue après 5000 itérations. Pourquoi ? Car le gradient en σ contient un terme en σ^{-3} , qui explose quand σ devient petit. Le gradient naturel, lui, atteint l'optimum à 10^{-6} près en 819 itérations ; la descente euclidienne n'y parvient jamais sur le budget testé.

On visualise l'effet sur la formule : $F^{-1} = \text{diag}(\sigma^2, \sigma^2/2)$ annule deux des trois puissances de σ au dénominateur. La métrique atténue le pas quand elle « sent » qu'un déplacement est hypersensible pour la loi, et inversement.

L'invariance par reparamétrisation

Pour mieux comprendre, on réécrit le même problème en coordonnées (μ, γ) avec $\gamma = \log \sigma$. C'est la même famille de gaussiennes, seul l'étiquetage change. Un algorithme qui dépend de ce choix dépend d'une convention arbitraire.

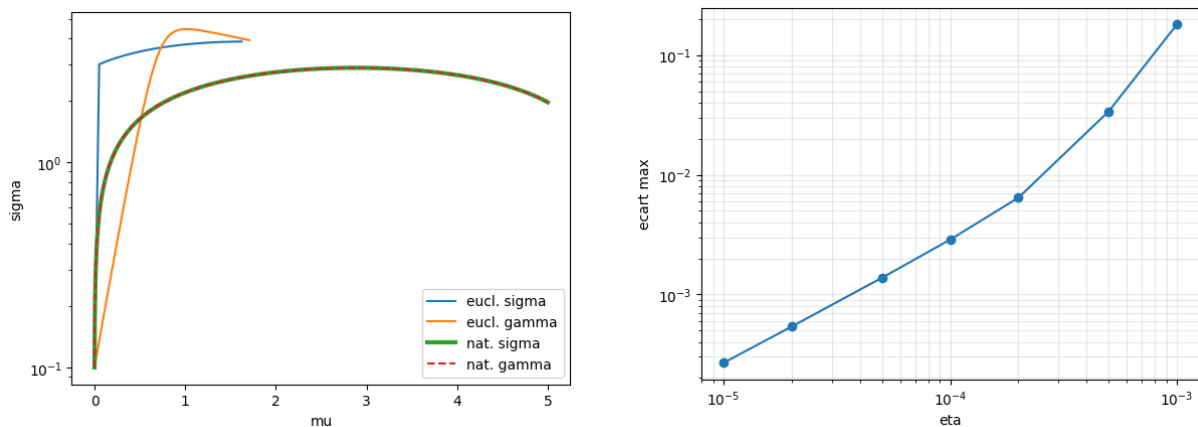


Figure 2: À gauche : quatre trajectoires, deux méthodes $\times$ deux paramétrisations, ramenées en coordonnées σ . Les deux naturelles se confondent. À droite : écart entre les deux trajectoires naturelles en fonction du pas, en échelle log-log.

On observe deux comportements : les trajectoires de la descente classique s'écartent fortement, tandis que celles du gradient naturel se superposent exactement.

L'explication découle du changement de variable et de la jacobienne $J = \frac{\partial \theta}{\partial \gamma}$:

- Le gradient devient : $\nabla_\gamma \tilde{h} = J^\top \nabla_\theta h$.
- La matrice de Fisher se transforme en : $F_\gamma = J^\top F_\theta J$.

En formant le gradient naturel dans le nouveau repère :

$$\begin{aligned} F_\gamma^{-1} \nabla_\gamma \tilde{h} &= (J^\top F_\theta J)^{-1} (J^\top \nabla_\theta h) \\ &= J^{-1} F_\theta^{-1} (J^\top)^{-1} J^\top \nabla_\theta h \\ &= J^{-1} (F_\theta^{-1} \nabla_\theta h). \end{aligned}$$

Le facteur « déformant » $J^\top$ s'annule : le gradient naturel convertit proprement le déplacement via J^{-1} , garantissant la même trajectoire intrinsèque. Le gradient ordinaire, lui, applique $J^\top$ au lieu de J^{-1} : sa trajectoire dépend donc du choix des coordonnées, sauf dans le cas particulier d'une rotation pure ($J^\top = J^{-1}$).

Pour s'en convaincre, minimisons $h(\theta_1, \theta_2) = \frac{1}{2}(\theta_1^2 + \theta_2^2)$ depuis $(1, 1)$. Une rotation ($J^\top = J^{-1}$) conserve le pas ordinaire $\theta^{(1)} = (0,9 ; 0,9)$, tandis qu'un simple changement d'échelle sur θ_1 ($\gamma = \theta_1/2$ donc $J \neq J^{-\top}$) dévie la trajectoire vers $(0,6 ; 0,9)$: seul le gradient naturel neutralise ce changement.

L'invariance n'est toutefois exacte qu'à la limite continue. Sur un pas fini, l'erreur de troncature est $\mathcal{O}(\eta^2)$ par itération, et comme il faut $N = T/\eta$ itérations pour parcourir une distance fixée, l'écart cumulé est $\mathcal{O}(\eta)$. La figure de droite le confirme : l'écart décroît linéairement avec le pas, et les deux trajectoires coïncident donc exactement pour le flot continu. Je renvoie à la référence [1] pour plus de détails.

3 Un MLP jouet : la Fisher exacte

Le modèle est un perceptron à 27 paramètres (`Linear(2,4)`, `Tanh`, `Linear(4,3)`), softmax et entropie croisée, sur 120 points répartis en trois classes. La Fisher 27×27 est construite par Monte-Carlo : pour chaque point, un label est échantillonné sous le modèle, le gradient correspondant est calculé, et on moyenne les produits extérieurs.

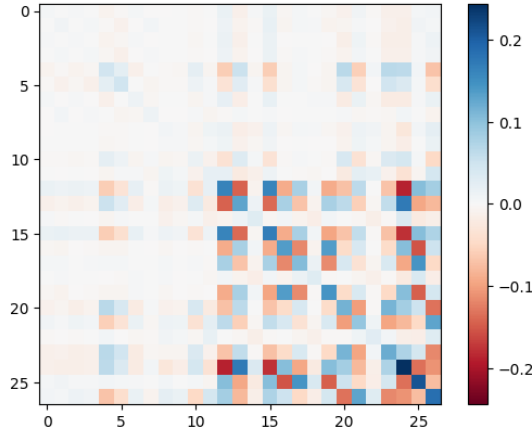


Figure 3: La matrice de Fisher exacte, 27×27 . Indices 0–11 : première couche. Indices 12–26 : seconde couche.

Elle est pleine, structurée en blocs par couche, avec des amplitudes bien plus fortes sur la seconde couche et des couplages non nuls entre les deux couches, contrairement à K-FAC que l'on verra plus tard.

Quels labels pour construire F ?

Par définition, il faut échantillonner y sous le modèle. En pratique, il serait bien plus tentant d'utiliser les vrais labels du jeu de données, déjà disponibles (et moins coûteux).

Cependant, il convient de ne pas utiliser les vrais labels : l'écart grandit avec l'entraînement. Sur le même modèle, les normes de la vraie Fisher et de la Fisher empirique valent 1,31 et 1,67 avant entraînement, puis 3,24 et 1,90 après. Le rapport s'inverse : ce ne sont pas deux estimations du même objet.

La théorie repose sur l'annulation du score moyen $\mathbb{E}_{y \sim p_\theta}[s] = 0$, issue de l'identité $\int \partial_\theta p_\theta = \partial_\theta \int p_\theta = 0$. Cette simplification exige impérativement d'échantillonner les labels selon la distribution prédite par le modèle p_θ .

Sous les vraies données $y^* \sim p_{\text{data}}$, cette annulation tombe : le Hessian de la perte fait apparaître un terme résiduel non nul,

$$\nabla_\theta^2 \mathcal{L}(\theta) = \underbrace{\mathbb{E}_{p_{\text{data}}} \left[\nabla_\theta \log p_\theta(y^*) \nabla_\theta \log p_\theta(y^*)^\top \right]}_{F_{\text{emp}}(\theta)} - \underbrace{\mathbb{E}_{p_{\text{data}}} \left[\frac{\nabla_\theta^2 p_\theta(y^*)}{p_\theta(y^*)} \right]}_{R(\theta) \neq 0}.$$

En négligeant ce terme résiduel $R(\theta)$ (trop lourd et coûteux car il demanderait le tenseur des dérivées secondes complètes), on fausse l'estimation de la courbure : F_{emp} finit par donner une construction qui est différente de celle de F .

Le problème : ce que le gradient naturel coûte

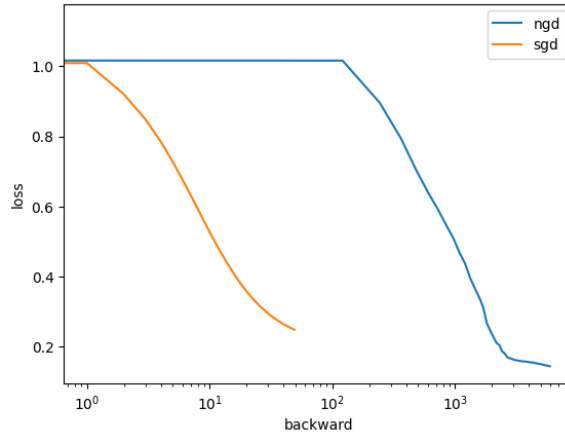


Figure 4: Gradient naturel exact contre SGD, en fonction du nombre de backwards. Chaque itération du gradient naturel en coûte 121, contre 1 pour SGD.

Le gradient naturel converge en bien moins d'itérations, mais est-ce rentable ?

La réponse est visiblement non, du moins pas sous cette forme. Par itération, le gradient naturel atteint une perte de 0,14 quand SGD est encore à 0,53. Mais construire F exactement demande un gradient *par point* : 120 passes arrière pour la métrique, plus une pour la direction. Mesuré en calcul plutôt qu'en itérations, SGD gagne largement.

C'est exactement le problème que K-FAC [3] doit résoudre.

4 K-FAC : approcher la métrique

Le principe

Le gradient d’une couche linéaire peut s’écrire en fonction des entrées et du gradient de sortie : si a est l’entrée de la couche et g le gradient par rapport à sa pré-activation de sortie, alors $\nabla_W L = g a^\top$. Aplati ligne par ligne, cela vaut $g \otimes a$, et le bloc de Fisher de la couche s’écrit exactement

$$F_{\text{bloc}} = \mathbb{E} \left[gg^\top \otimes aa^\top \right].$$

On rappelle bien sûr que $(A \otimes B)(A \otimes B)^\top = (AA^\top \otimes BB^\top)$ où $\otimes$ est le produit de Kronecker.

K-FAC fait alors deux approximations : il ignore les couplages entre couches (F devient bloc-diagonale), et il échange l’espérance et le produit de Kronecker,

$$\mathbb{E} \left[gg^\top \otimes aa^\top \right] \approx \underbrace{\mathbb{E}[gg^\top]}_G \otimes \underbrace{\mathbb{E}[aa^\top]}_A.$$

Le gain vient de $(G \otimes A)^{-1} = G^{-1} \otimes A^{-1}$, et l’action sur le gradient d’une couche se réduit à deux produits matriciels :

$$\Delta = -\eta G^{-1} V A^{-1}, \quad V = [\nabla_W L \mid \nabla_b L].$$

On n’inverse jamais le bloc complet, seulement deux matrices de tailles n_{out} et $n_{\text{in}} + 1$. Le biais est absorbé en ajoutant un 1 au vecteur d’activation.

L’approximation est-elle bonne ?

Sur un modèle assez petit pour que F soit calculable, ce que nous voulons savoir est : de combien K-FAC s’écarte-t-il de la vérité ? C’est une mesure qu’on ne pourra plus faire à grande échelle.

Cela dépend fortement de la couche :

Situation	Erreur relative sur le bloc
Un seul point de données	$3,4 \times 10^{-9}$
120 points, couche d’entrée	222 %
120 points, couche après tanh	6 %

Sur un point unique il n’y a rien à moyenner et la factorisation est exacte : ce régime sert de test de correction de l’implémentation. L’erreur ne vient donc que de l’échange espérance/Kronecker.

Sur la couche d’entrée, K-FAC surestime la courbure d’un facteur 2,8 en norme. La trace du produit de Kronecker vaut $\mathbb{E}\|g\|^2 \mathbb{E}\|a\|^2$ alors que la trace exacte vaut $\mathbb{E}[\|g\|^2 \|a\|^2]$: quand les deux quantités varient en sens opposé, le produit des moyennes dépasse la moyenne du produit. C’est le cas sur des données brutes réparties en trois groupes très séparés. Après le tanh, les activations sont bornées dans $[-1, 1]$ et bien plus homogènes, et l’approximation devient très bonne.

Le problème de coût

Construire A et G ne demande pas un gradient par point. Les hooks interceptent a et g avant que PyTorch ne les agrège, si bien qu’une seule passe arrière sur tout le lot suffit [4] :

$$A = \frac{1}{N} A_{\text{batch}}^\top A_{\text{batch}} \quad \text{et} \quad G = \frac{1}{N} G_{\text{batch}}^\top G_{\text{batch}}.$$

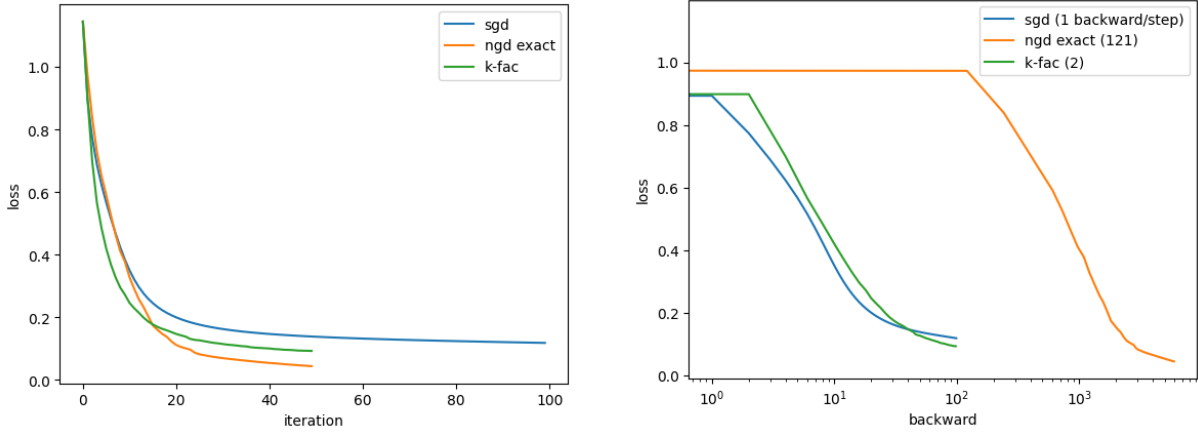


Figure 5: Sur le MLP. À gauche, par itération : K-FAC suit de près le gradient naturel exact. À droite, en nombre de passes arrière, avec SGD réglé à son meilleur taux.

La version groupée coïncide avec la version point par point à 10^{-7} près, et le coût tombe de 121 à 2 backwards par itération.

Ainsi K-FAC est-il alors préférable à SGD ?

Sur ce problème, à priori non. Par itération, K-FAC reproduit bien le gradient naturel exact, pour un coût proche de celui de SGD : l’objectif de diminuer le coût est vérifié. Mais à budget égal, un SGD correctement réglé atteint le même niveau. Un premier essai laissait croire le contraire, avec un avantage net pour K-FAC ; cet avantage venait d’un SGD sous-réglé, et disparaît une fois son taux d’apprentissage optimisé.

L’interprétation est que le problème est trop bien conditionné pour que la géométrie apporte quelque chose : 27 paramètres, trois groupes bien séparés. Or le gradient naturel sert précisément à corriger un mauvais conditionnement.

5 Le transformeur

Le modèle est un transformeur [5] caractère par caractère entraîné sur tinyshakespeare, dans une configuration réduite (4 couches, 4 têtes, $n_{\text{embd}} = 128$, contexte 64, soit 0,82 M de paramètres et 61 couches linéaires), testé sur GPU T4 avec un batch de 32 séquences. Chaque couche linéaire étant appliquée aux 64 positions de la séquence avec les mêmes poids, chaque position est traitée comme un point de données : un lot fournit $32 \times 64 = 2048$ couples (a, g) par couche. Les embeddings et les normalisations, qui n’ont pas la structure $ga^\top$, sont confiés à AdamW.

Deux réglages pratiques se sont révélés nécessaires. Les facteurs ne sont recalculés qu’une itération sur dix (lissés par moyenne glissante exponentielle à $\beta = 0,95$), et un momentum $\mu = 0,9$ est ajouté au pas K-FAC : sans lui, la comparaison serait « déloyale », puisque AdamW et SGD en ont un. Chaque méthode a ensuite été réglée séparément à son meilleur.

Méthode	Réglage	Perte val.	Temps
AdamW	$\text{lr} = 3 \cdot 10^{-4}$	1,905	34,1 s
SGD + momentum	$\text{lr} = 0,1, \mu = 0,9$	1,948	32,1 s
K-FAC	$\eta = 0,1, \lambda = 1, \mu = 0,9$	1,917	59,6 s

Ainsi, sur un problème réellement mal conditionné, K-FAC prend-il l’avantage ?

En nombre d’itérations, la direction reste très qualitative. En temps de calcul réel, non : sa

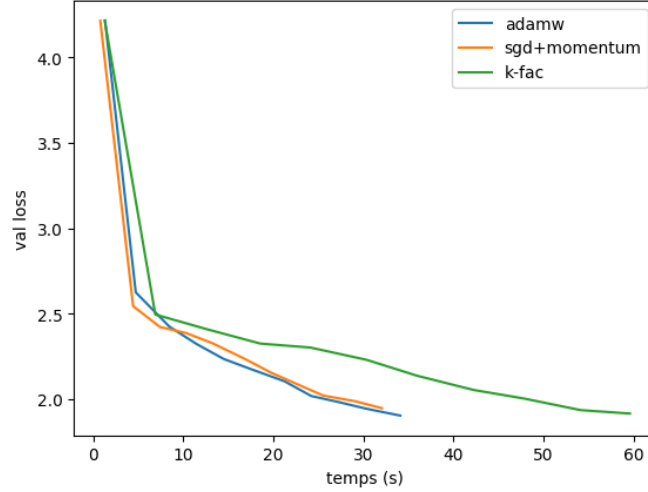


Figure 6: Perte de validation en fonction du temps écoulé, 1000 itérations, même initialisation.

la courbe reste au-dessus de celle d’AdamW sur la totalité du parcours, et il lui faut presque deux fois plus de temps pour atteindre un palier similaire. Augmenter l’intervalle de rafraîchissement des facteurs de 10 à 50 ne change pas le verdict (1,92 en 60 s) : le goulot n’est pas seulement le calcul des facteurs, mais aussi les 122 inversions et produits matriciels par pas, sur 61 couches.

Le problème est bien mal conditionné, cependant : SGD sans momentum reste bloqué 400 itérations sur un plateau autour de 3,3, que ni AdamW ni K-FAC ne rencontrent.

Une limite identifiée : le damping

Le damping λ est le même pour les 61 couches, alors que leurs facteurs n’ont pas du tout les mêmes échelles :

$$A_{\lambda}^{-1} = (A + \lambda I)^{-1}, \quad G_{\lambda}^{-1} = (G + \lambda I)^{-1}.$$

Seul un λ grand ($\lambda = 1$) donne un entraînement stable ; avec $\lambda = 0,1$, la perte explose. Or un λ grand écrase la géométrie précisément dans les couches où G est petite, puisque $G + \lambda I \approx \lambda I$. K-FAC n’y exprime donc plus grand-chose. Un damping adapté couche par couche, proportionné à la taille des facteurs, est la première piste à explorer pour que la méthode donne sa pleine mesure.

6 Conclusion

Ce court projet peut se résumer de la façon suivante : la géométrie compte. Sur un modèle gaussien, elle fait la différence entre une méthode qui diverge et une qui converge en 819 itérations, et elle rend la descente indépendante du choix de coordonnées. Mais elle coûte cher : la Fisher exacte demande un gradient par point, ce qui la rend impraticable. K-FAC ramène ce coût à deux backwards par itération, au prix d’une approximation dont on a mesuré la qualité : excellente après une non-linéarité bornée, mauvaise sur des entrées brutes.

Au bout du compte, sur les deux modèles testés, K-FAC gagne en nombre d’itérations et perd en temps réel face à un optimiseur du premier ordre correctement réglé. C’est un résultat négatif, mais mesuré des deux façons. La méthode n’échoue pas par manque de qualité de direction, elle échoue par son coût par pas.

Finalement, trois réserves sur la portée de mes conclusions. Les expériences portent sur la configuration réduite du transformeur ; le passage à 10,8 M de paramètres n’a pas été mené

(trop long pour mon ordinateur), et rien ne garantit que l'arbitrage soit le même à cette échelle, où le conditionnement est probablement plus mauvais. Les écarts finaux entre méthodes sont faibles et mériteraient d'être confirmés sur plusieurs initialisations. Enfin, l'implémentation est directe et non optimisée.

Les pistes naturelles sont donc le damping par couche, un rafraîchissement des facteurs plus espacé encore avec inversion amortie, et le passage à la configuration complète du transformeur. Je creuse encore cette thématique que je ne maîtrise pas complètement, donc n'hésitez pas à me corriger ou échanger sur ce sujet.

References

- [1] J. Martens, *New insights and perspectives on the natural gradient method*, arXiv:1412.1193, 2014.
- [2] S.-I. Amari, *Natural gradient works efficiently in learning*, Neural Computation, 10(2):251–276, 1998.
- [3] J. Martens et R. Grosse, *Optimizing neural networks with Kronecker-factored approximate curvature*, International Conference on Machine Learning (ICML), 2015.
- [4] F. Dangel, B. Mucsányi, T. Weber et R. Eschenhagen, *Kronecker-factored Approximate Curvature (KFAC) from Scratch*, arXiv:2507.05127, 2025.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser et I. Polosukhin, *Attention is all you need*, Advances in Neural Information Processing Systems (NeurIPS), 2017.